

Laborator 4

Fișiere

4.1 Scopul lucrării

Laboratorul curent prezintă principalele apeluri de sistem în Linux pentru manipularea fișierelor.

4.2 Fundamente teoretice

4.2.1 Conceptul de *file descriptor*

Sistemul de operare asignează fiecărui fișier deschis un descriptor sau identificator, reprezentat ca un întreg. De fiecare dată când se face un apel de sistem ce creează sau deschide un fișier, sistemul de operare returnează un astfel de descriptor. Descriptorii de fișiere sunt unici la nivel de proces (program).

Prin convenție, primii 3 descriptori de fișiere sunt standard:

- 0: *standard input* (ex.: funcția `scanf` citește din standard input);
- 1: *standard output* (ex.: funcția `printf` scrie în standard output);
- 2: *standard error* (ex.: funcția `perror` scrie în standard error).

Există cinci apeluri de sistem ce creează descriptori: `creat`, `open`, `fcntl`, `dup` și `pipe`.

4.2.2 Crearea, deschiderea și închiderea fișierelor

```
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
#include <unistd.h>

int open(const char *pathname, int flags);
int open(const char *pathname, int flags, mode_t mode);

int creat(const char *pathname, mode_t mode);

int close(int fd);
```

4.2.2.1 open

Pentru a deschide un fișier se folosește apelul de sistem `open`. În prima variantă (cu doi parametri), primul parametru reprezintă numele fișierului sau calea spre acesta (absolută sau relativă la folderul curent), iar al doilea indică felul în care vrem să deschidem fișierul (citire, scriere, etc.). Al treilea parametru se folosește doar la crearea unui fișier nou și indică permisiunile de acces.

Argumentul `flags` reprezintă o mască de biți (fiecare bit setat indică un anumit *flag*), ce poate fi o combinație între următoarele valori (definite în `fcntl.h`):

- `O_RDONLY` - deschide fișierul pentru citire;
- `O_WRONLY` - deschide fișierul pentru scriere;
- `O_RDWR` - deschide fișierul pentru citire și scriere;
- `O_APPEND` - deschide fișierul pentru scriere, dar dacă avem ceva scris în el, se va scrie doar după sfârșitul fișierului, prin adăugare;
- `O_CREAT` - în cazul în care fișierul nu există deja, este creat;
- `O_EXCL` - dacă folosim `O_CREAT` și fișierul există deja, apelul de sistem va eșua;
- `O_TRUNC` - dacă fișierul are deja un conținut, acesta se va șterge.

Există și alte flag-uri ce pot fi folosite la deschiderea unui fișier. Pentru o listă completă a lor, precum și pentru alte detalii legate de apelul de sistem `open`, utilizați comanda `man 2 open`.

În cazul folosirii argumentului `mode`, acesta indică permisiunile fișierului creat și este construit tot ca o mască de biți, prin combinarea (SAU pe biți) următoarelor valori (definite în `sys/stat.h`):

- `S_IRUSR`, `S_IWUSR`, `S_IXUSR` - *read*, *write*, *execute* pentru *owner*-ul fișierului;
- `S_IRGRP`, `S_IWGRP`, `S_IXGRP` - *read*, *write*, *execute* pentru membrii grupului din care face parte *owner*-ul fișierului;
- `S_IROTH`, `S_IWOTH`, `S_IXOTH` - *read*, *write*, *execute* pentru restul utilizatorilor.

Alternativ, se poate folosi pentru `mode` reprezentarea ca număr octal a permisiunilor de acces, ca de exemplu `0644`.

Rezultatul returnat este un descriptor de fișier valid, în caz de succes, sau valoarea `-1`, în caz de eșec.

4.2.2.2 creat

Apelul de sistem `creat` este utilizat pentru a crea un fișier și este similar cu a apela `open`, cu flag-urile `O_WRONLY` | `O_CREAT` | `O_TRUNC`. Argumentele `pathname` și `mode` au aceeași semnificație și aceleași valori posibile ca și la `open`.

Valoarea returnată este, de asemenea, un descriptor de fișier valid, în caz de succes sau `-1`, în caz de eșec.

4.2.2.3 close

Apelul de sistem `close` realizează închiderea fișierului al cărui descriptor a fost primit ca parametru. Va returna `0`, în caz de succes și `-1`, în caz de eșec.

4.2.2.4 Exemplu

Dorim să scriem un program care face o copie a fișierului `/usr/include/stdlib.h` în folderul curent. Vom începe prin a deschide și închide fișierele, utilizând apelurile de sistem prezentate mai sus.

```

1 | int main()
2 | {
3 |     int fd1 = -1, fd2 = -1;
4 |
5 |     fd1 = open("/usr/include/stdlib.h", O_RDONLY);
6 |     if(fd1 == -1) {
7 |         perror("Could not open input file");
8 |         return 1;
9 |     }
10 | }
```

```

11     fd2 = creat("copy_stdlib.h", S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH);
12     if(fd2 == -1) {
13         perror("Could not open output file");
14         close(fd1);
15         return 1;
16     }
17
18     ...
19
20     close(fd1);
21     close(fd2);
22
23     return 0;
24 }

```

Programul începe prin a declara variabilele `fd1` și `fd2` pentru cei doi descriptori de fișier (fișierul de intrare și fișierul de ieșire), la linia 3.

La linia 5 se deschide fișierul de intrare pentru citire, folosind flag-ul `O_RDONLY`. La linia 11 se deschide fișierul de ieșire pentru scriere. Deoarece e posibil ca acest fișier să nu existe încă, vom folosi apelul de sistem `creat` în loc de `open` (se putea deschide și cu `open`, folosind flag-urile `O_WRONLY`, `O_CREAT` și `O_TRUNC`).

După fiecare deschidere de fișier este important să verificăm valoarea descriptorului de fișier. Valoarea `-1` indică eșecul. În acest caz, operația curentă nu mai poate continua. Înainte de ieșire vom afișa un mesaj de eroare folosind funcția `perror` (liniile 7 și 13), iar dacă avem deja descriptori de fișier deschiși se vor închide (linia 14).

Funcția `perror` va afișa string-ul primit ca argument, urmat de motivul ultimei erori. De exemplu, dacă fișierul de intrare nu ar exista, apelul de `open` de la linia 5 ar returna `-1`, iar la linia 7 s-ar afișa “Could not open input file: No such file or directory”.

Înainte de finalul programului, este important să închidem descriptorii de fișiere utilizați, utilizând apelul de sistem `close` (liniile 20 și 21).

Pentru a realiza copierea, nu este suficient să deschidem fișierele. În secțiunea următoare vom studia apelurile de sistem necesare pentru citirea și scrierea unui fișier deschis.

4.2.3 Citirea și scrierea

```

#include <unistd.h>

ssize_t read(int fd, void *buf, size_t count);
ssize_t write(int fd, const void *buf, size_t count);

```

4.2.3.1 read

Apelul de sistem `read` citește de la poziția curentă din fișierul deschis `fd` un număr `count` de octeți pe care îi va scrie în buffer-ul `buf`. După citire, poziția curentă în fișier va avansa cu numărul de octeți citați.

Apelul `read` va returna numărul de octeți citați în caz de succes sau valoarea `-1` în caz de eroare. Se poate întâmpla ca numărul de octeți citați să nu corespundă cu valoarea parametrului `count`. De exemplu, putem încerca să citim 8 octeți, dar până la finalul fișierului să mai fie doar 3. În acest caz, `read` va returna valoarea 3. Dacă se returnează valoarea 0, înseamnă că poziția curentă în fișier era la sfârșitul fișierului înaintea execuției apelului `read` și, prin urmare, nu s-a citit nimic.

4.2.3.2 write

Apelul de sistem `write` scrie în fișierul deschis `fd` un număr `count` de octeți din buffer-ul `buf`. După scriere, poziția curentă va avansa cu numărul de octeți scriși.

Apelul `write` va returna numărul de octeți scriși în caz de succes sau valoarea `-1` în caz de eroare. Dacă se returnează o valoare mai mică decât parametrul `count`, cel mai probabil discul e plin și nu s-au putut scrie toate datele.

4.2.3.3 Exemplu

Vom continua exemplul din secțiunea 4.2.2.4, adăugând cod pentru copierea conținutului din primul fișier în al doilea.

```

1  #define BUFF_SIZE 64
2
3  int main()
4  {
5      int fd1 = -1, fd2 = -1;
6      ssize_t size = 0;
7      int err = 0;
8      unsigned char buff[BUFF_SIZE];
9
10     ... //the code for opening the files
11
12     for(;;) {
13         size = read(fd1, buff, BUFF_SIZE);
14         if(size < 0) {
15             //read error
16             perror("Could not read from input file");
17             err = 1;
18             break;
19         } else if(size == 0) {
20             //end of file
21             break;
22         } else {
23             //size may be smaller than BUFF_SIZE
24             size = write(fd2, buff, size);
25             if(size <= 0){
26                 perror("Could not write to output file");
27                 err = 2;
28                 break;
29             }
30         }
31     }
32
33     ... //the code for closing the files
34
35     if(err == 0) {
36         printf("File copied successfully.\n");
37     } else {
38         printf("Some error occurred while copying.\n");
39     }
40
41     return 0;
42 }
```

Copierea propriu-zisă a conținutului se face în bucla de la liniile 12–31. La fiecare iterație, se citește un bloc de dimensiune `BUFF_SIZE` din primul fișier (linia 13), bloc care se scrie în al doilea fișier (linia 24).

La ambele apeluri de sistem, dacă valoarea returnată este negativă vom afișa un mesaj de eroare, vom seta o variabilă care să indice eroarea, apoi vom părăsi bucla (liniile 15–18 și 26–28). În mod normal, se va ieși din buclă prin `break`-ul de la linia 21, atunci când se ajunge la finalul fișierului de intrare, iar `read` returnează 0.

4.2.4 Deplasarea cursorului în fișiere

Fiecărui fișier deschis îi corespunde un cursor care indică poziția curentă, de unde se va citi la următorul `read` sau se va scrie la următorul `write`. Poziția cursorului este incrementată implicit în urma operațiilor de `read` și `write` sau poate fi modificată explicit prin apelul sistem `lseek`.

```
#include <sys/types.h>
#include <unistd.h>

off_t lseek(int fd, off_t offset, int whence);
```

Apelul de sistem `lseek` primește ca argument un descriptor de fișier `fd`, un deplasament `offset` (pozitiv sau negativ) și un indicator `whence`, care spune de unde se calculează deplasamentul. Parametrul `whence` poate lua următoarele valori:

- `SEEK_SET` - deplasamentul se consideră de la începutul fișierului;
- `SEEK_CUR` - deplasamentul se consideră de la poziția curentă;
- `SEEK_END` - deplasamentul se consideră de la finalul fișierului.

Valoarea returnată reprezintă poziția absolută a cursorului în fișier.

4.2.4.1 Exemplu: determinarea dimensiunii unui fișier

Ideea de bază pentru a determina dimensiunea unui fișier (numărul de octeți din acesta) este să ne deplasăm la finalul și să aflăm poziția cursorului.

```
1  ...
2  int fd;
3  off_t size;
4
5  fd = open(...);
6  if(fd == -1) { ... }
7
8  size = lseek(fd, 0, SEEK_END);
9  printf("The file size is %ld.\n", size);
10 ...
```

Codul de mai sus începe prin a deschide fișierul și a verifica dacă deschiderea s-a efectuat cu succes (liniile 5–6). Apelul de sistem către `lseek` se face la linia 8, unde se cere deplasarea cursorului exact la finalul fișierului. Apelul returnează poziția cursorului în fișier după deplasare, adică exact poziția de la final.

Tipul de date `off_t` depinde de platformă (trebuie să fie suficient de mare ca să poată indica o poziție într-un fișier), de obicei fiind definit ca un `long int`. Din acest motiv, la afișare folosim formatul `%ld`.

4.2.4.2 Exemplu: copierea cifrelor la finalul unui fișier

Dorim să scriem un program care citește un fișier caracter cu caracter, scriind cifrele la finalul fișierului.

```
1  int fd;
2  off_t size, crtPos, i;
3  char c = 0;
4  ...
5  fd = open(..., O_RDWR);
6  if(fd == -1) { ... }
7
8  size = lseek(fd, 0, SEEK_END);
9  lseek(fd, 0, SEEK_SET);
```

```

10
11     for(i=0; i<size; i++) {
12         if(read(fd, &c, 1) != 1) {
13             perror("Reading error");
14             break;
15         }
16         if(c >= '0' && c <= '9') {
17             crtPos = lseek(fd, 0, SEEK_CUR);
18             lseek(fd, 0, SEEK_END);
19             if(write(fd, &c, 1) != 1) {
20                 perror("Writing error");
21                 break;
22             }
23             lseek(fd, crtPos, SEEK_SET);
24         }
25     }
26
27     close(fd);

```

Ca de obicei, se deschide întâi fișierul și se verifică dacă deschiderea s-a făcut cu succes (liniile 5–6). În acest exemplu vom folosi pentru deschidere flag-ul `O_RDWR`, deoarece dorim să efectuăm atât operațiile de citire, cât și cele de scriere pe același fișier.

La linia 8, determinăm dimensiunea inițială a fișierului, ca și în exemplul anterior, apoi readucem cursorul la începutul fișierului (linia 9).

Deoarece se cunoaște dimensiunea fișierului, putem realiza citirea într-o buclă cu număr fix de pași (liniile 11–25). La linia 12 se citește un singur caracter, direct în variabila `c`. Dacă caracterul citit este cifră (conform verificării de la linia 16), vom efectua următorii pași:

- salvăm poziția curentă în variabila `crtPos` (linia 17), pentru că după ce se scrie cifra la finalul fișierului, trebuie să știm unde ne vom întoarce pentru a citi următorul caracter;
- mutăm cursorul la finalul fișierului (linia 18);
- scriem cifra citită (linia 19) și verificăm dacă scrierea s-a efectuat cu succes;
- mutăm cursorul înapoi pe poziția `crtPos` (linia 23), pregătind fișierul pentru următoarea citire.

4.3 Mersul lucrării

4.3.1 Întrebări recapitulative

1. Ce apel de sistem folosește intern funcția `printf`?
2. Ce apeluri de sistem se pot folosi pentru a crea un fișier?
3. Ce returnează apelurile de sistem `read` și `write`?
4. Cum aflăm dimensiunea unui fișier deja deschis, utilizând `lseek`?
5. Presupunând că fișierul `file.txt` are dimensiunea 100 de octeți iar buffer-ul `buff` 50 de octeți, care va fi dimensiunea fișierului după ce se execută codul de mai jos?

```

1 | int fd = open("file.txt", O_WRONLY);
2 | lseek(fd, 80, SEEK_SET);
3 | write(fd, buff, 50);
4 | close(fd);

```

4.3.2 Probleme rezolvate

1. (*l04p1_copy.c*) Să se scrie un program care face o copie, în folderul curent, a fișierului `/usr/include/stdlib.h`. Programul este descris în secțiunile 4.2.2.4 și 4.2.3.3.
2. (*l04p2_size.c*) Să se scrie un program care primește ca argument numele sau calea unui fișier și afișează dimensiunea acestuia în octeți. Programul este descris în secțiunea 4.2.4.1.
3. (*l04p3_digits.c*) Să se scrie un program care primește ca argument numele sau calea unui fișier și copiază la finalul fișierului toate cifrele găsite în el. Programul este descris în secțiunea 4.2.4.2.

4.3.3 Probleme propuse

1. (25p) Implementați funcția

```
int get_line(int fd, int lineNr, char *line, int maxLength);
```

care citește linia cu numărul `lineNr` din fișierul deschis cu descriptorul `fd`. Pentru a delimita liniile din fișier, țineți cont că acestea se termină cu caracterul ‘`\n`’. Buffer-ul `line` în care se va scrie linia citită are dimensiunea `maxLength`, în care trebuie să încapă inclusiv terminatorul de string (un octet cu valoarea 0). Funcția va returna 0 în caz că citirea s-a efectuat cu succes și valori negative pentru cazurile de eroare (erori la citire, linie prea lungă, fișierul are mai puțin de `lineNr` linii, etc.). Scrieți de asemenea un program care primește ca argument un nume de fișier și un număr de linie și apelează funcția `get_line`.
2. (25p) Scrieți un program care citește un fișier și construiește un alt fișier cu conținutul inversat. Numele fișierului sursă și al fișierului destinație se primesc ca argumente în linia de comandă.
3. (25p) Scrieți un program care citește un fișier și construiește un alt fișier cu liniile inversate (prima linie se inversează cu ultima, a doua cu penultima, etc.). Pentru punctaj maxim trebuie ținut cont de faptul că o linie poate fi oricât de lungă. Numele fișierului sursă și al fișierului destinație se primesc ca argumente în linia de comandă.
4. (25p) Scrieți un program care inserează un string într-un fișier, pe o poziție dată. De exemplu, dacă fișierul are conținutul `abcdefghijkl` și se dorește inserarea string-ului ‘`XYZ`’ pe poziția 4, fișierul va avea conținutul `abcdXYZefghijkl`.

